NLP 243 Deep Learning for Natural Language Processing

Fall 2024

Outline for Today

- Projects
- From Linear Regression to Neural Networks
- Practical Tips
- Linear Neural Networks for Classification
- Activation and Loss Functions using PyTorch
- Readings:
  - Ch2 of NLP with PyTorch
  - Ch3 of NLP with PyTorch
  - Ch3 of Dive into Deep Learning
  - Ch5 of Dive into Deep Learning

Class Project: Apply Existing NNets to Tasks

1. Define Task:
   - Summarization
   - Named Entity Recognition
   - Sentence Similarity

2. Define Dataset
   - Search for academic datasets
   - They already have baselines
   - E.g: Document Understanding Conference (DUC) for Summarization

Class Project: Apply Existing NNets to Tasks

3. Define your metric
   - Search online for well established metrics on this task
   - Summarization: Rouge (Recall-Oriented Understudy for Gisting Evaluation) which defines n-gram overlap to human summaries

4. Split your dataset!
   - Train/Dev/TesT
   - Academic dataset often come pre-split
   - Don’t look at the test split until ~1 week before deadline! (or at most once a week)

Class Project: Apply Existing NNets to Tasks

5. Establish a baseline
   - Implement the simplest model (often logistic regression on unigrams and bigrams) first
   - Compute metrics on train AND dev
     - Analyze errors
     - If metrics are amazing and no errors: done, problem was too easy, restart :)

6. Implement existing neural net model
   - Compute metric on train and dev
   - Analyze output and errors
   - Minimum bar for this class

Class Project: Apply Existing NNets to Tasks

7. Visualize the dataset
   - Collect summary statistics
   - Look at errors
   - Analyze how different hyperparameters affect performance

8. Try out different model variants
   - Word vector averaging model (neural bag of words)
   - Convolutional neural network
   - Recurrent neural network
   - Transformer Networks

Named Entity Recognition (NER)

- Named entities are phrases that contain the names of persons, organizations, locations, times and quantities. Example:

  [ORG U.N.] official [PER Ekeus] heads for [LOC Baghdad] .

- CoNLL 2003, language independent NER.
- Data from English and German newspaper articles.

Machine Reading Comprehension (MRC)

- **S**tanford **Qu**estion **A**nswering **D**ataset (SQuAD)

The first recorded travels by Europeans to China and back date from this time. The most famous traveler of the period was the Venetian Marco Polo, whose account of his trip to "Cambaluc," the capital of the Great Khan, and of life there astounded the people of Europe. The account of his travels, Il milione (or, The Million, known in English as the Travels of Marco Polo), appeared about the year 1299. Some argue over the accuracy of Marco Polo's accounts due to the lack of mentioning the Great Wall of China, tea houses, which would have been a prominent sight since Europeans had yet to adopt a tea culture, as well the practice of foot binding by the women in capital of the Great Khan. Some suggest that Marco Polo acquired much of his knowledge through contact with Persian traders since many of the places he named were in Persian.

How did some suspect that Polo learned about China instead of by actually visiting it?

Answer: through contact with Persian traders

Possible Approaches:

- Simple classifiers for answer sentence selection and then parsing to extract the answer.
- Neural networks with attention.
- Combination of the above.

Conversational QA

Similar to SQUAD, but a sequence of questions.

[CoQA]

[QuAC]

Possible Approaches:

- Similar ideas to MRC
- Coreference resolution at each step, and then similar ideas to MRC.

The Virginia governor's race, billed as the marquee battle of an otherwise anticlimactic 2013 election cycle, is shaping up to be a foregone conclusion. Democrat Terry McAuliffe, the longtime political fixer and moneyman, hasn't trailed in a poll since May. Barring a political miracle, Republican Ken Cuccinelli will be delivering a concession speech on Tuesday evening in Richmond. In recent ...

Q_1: What are the candidates **running** for?
A_1: Governor
R_1: The Virginia governor's race

Q_2: **Where**?
A_2: Virginia
R_2: The Virginia governor's race

Q_3: Who is the democratic candidate?
A_3: **Terry McAuliffe**
R_3: Democrat Terry McAuliffe

Q_4: Who is **his** opponent?
A_4: **Ken Cuccinelli**
R_4 Republican Ken Cuccinelli

Q_5: What party does **he** belong to?
A_5: Republican
R_5: Republican Ken Cuccinelli

Q_6: Which of **them** is winning?
A_6: Terry McAuliffe
R_6: Democrat Terry McAuliffe, the longtime political fixer and moneyman, hasn't trailed in a poll since May

Other NLP tasks

- One of more of GLUE tasks. Paper
- Single sentence tasks (i.e., sentiment classification)
- Similarity and Paraphrase tasks
- Inference tasks (i.e., relationship between two sentences, entailment, contradiction, neutral)

GLUE Tasks

Name	Download	More Info	Metric
The Corpus of Linguistic Acceptability			Matthew's Corr
The Stanford Sentiment Treebank			Accuracy
Microsoft Research Paraphrase Corpus			F1 / Accuracy
Semantic Textual Similarity Benchmark			Pearson-Spearman Corr
Quora Question Pairs			F1 / Accuracy
MultiNLI Matched			Accuracy
MultiNLI Mismatched			Accuracy
Question NLI			Accuracy
Recognizing Textual Entailment			Accuracy
Winograd NLI			Accuracy
Diagnostics Main			Matthew's Corr

Possible approaches for NLI Tasks

- Task: Given 2 sentences, do they entail or contradict each other or neither?
- You can learn a representation for each sentence (i.e., CNN or RNN), then use a similarity approach to determine entailment or not.
- You can also learn a classifier that uses the two representations and decides on one of the classes.
- Triplet loss
- Siamese network

Other NLP Tasks

- One or more of SuperGLUE tasks

Name	Identifier	Download	More Info	Metric
Broadcoverage Diagnostics	AX-b			Matthew's Corr
CommitmentBank	CB			Avg. F1 / Accuracy
Choice of Plausible Alternatives	COPA			Accuracy
Multi-Sentence Reading Comprehension	MultiRC			F1a / EM
Recognizing Textual Entailment	RTE			Accuracy
Words in Context	WiC			Accuracy
The Winograd Schema Challenge	WSC			Accuracy
BoolQ	BoolQ			Accuracy
Reading Comprehension with Commonsense Reasoning	ReCoRD			F1 / Accuracy
Winogender Schema Diagnostics	AX-g			Gender Parity / Accuracy

Final Project Teams Sign Up

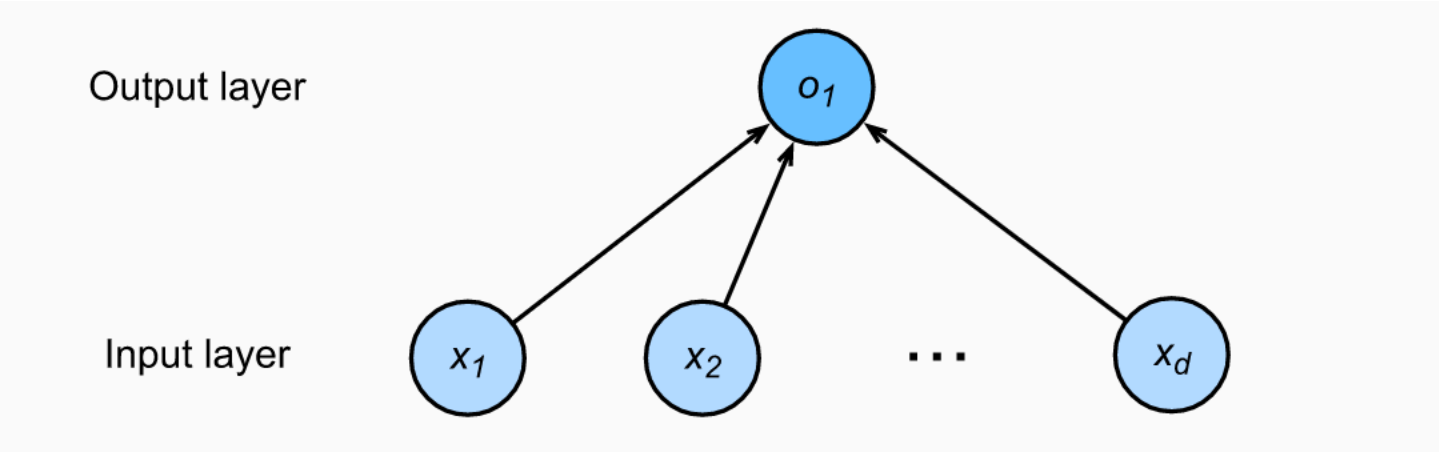
- October 17th
- Send a note to Sicong to sign up your team (and list members)
- Let us know earlier if you cannot find team members, we can help with teaming up with others.

Final Project Proposals

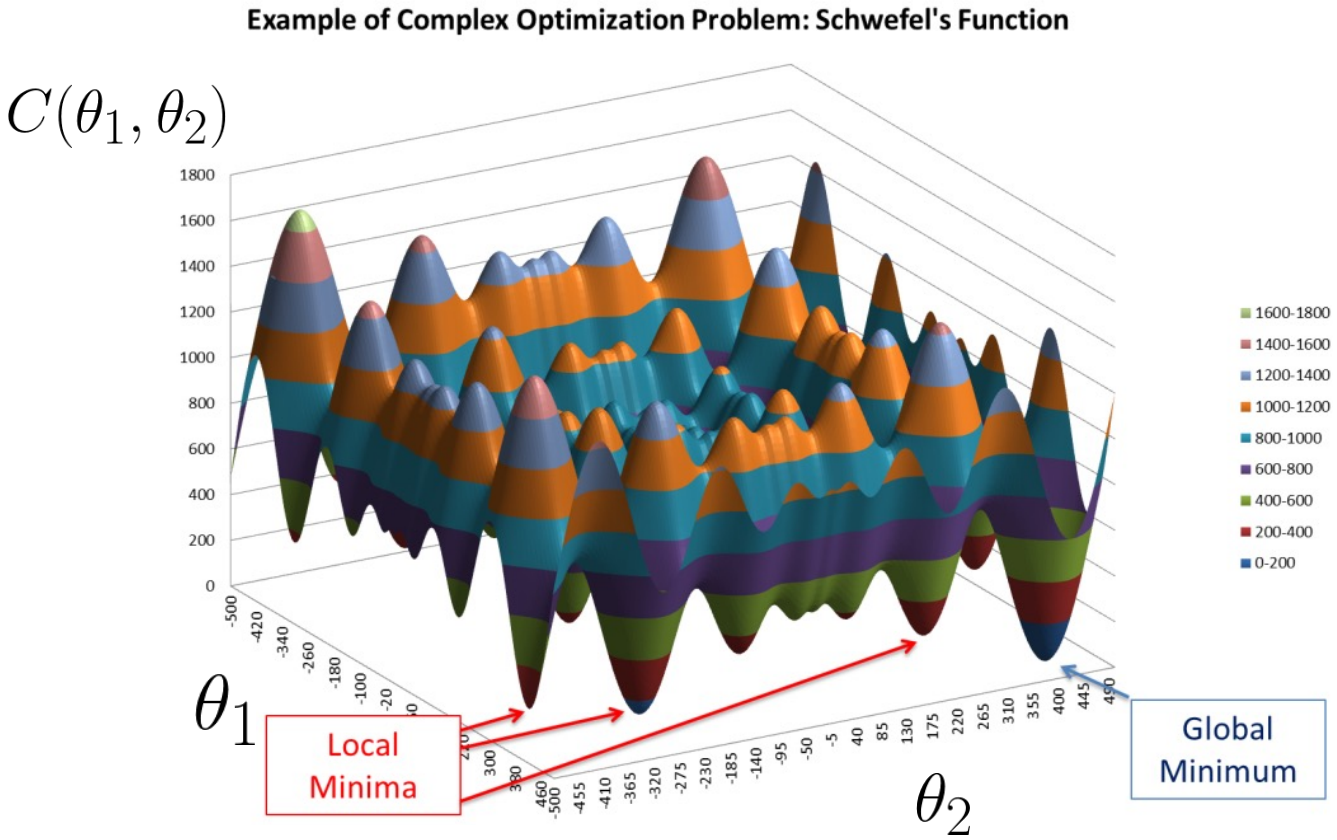
- **Oct 31th:** Final Project Proposal Presentations: In class presentations
  - 7 minute presentation
  - 2 minute Q&A
- **Oct 31th:** Final Project Proposal Report (2 pages)
- Both the presentation and report should include:
  - What is the NLP task you chose?
  - What is the motivation for the task?
  - Brief review of 2 papers that are about this task and your proposed approach
  - What is the dataset?
  - How do you plan to measure progress?

From Linear Regression to Neural Networks

- Linear regression is a single-layer neural network, consisting of just a single neuron!
- Fully connected layer (also called *dense* layer): every input is connected to every output.



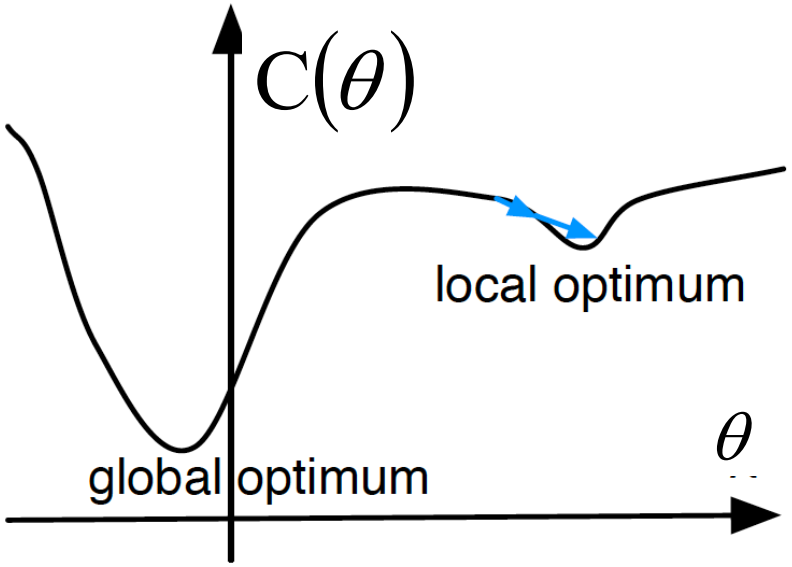
Big Issue: Local Optima



Neural networks have <u>no guarantee</u> for obtaining the global optimal solution!

Practical Tips: Initialization

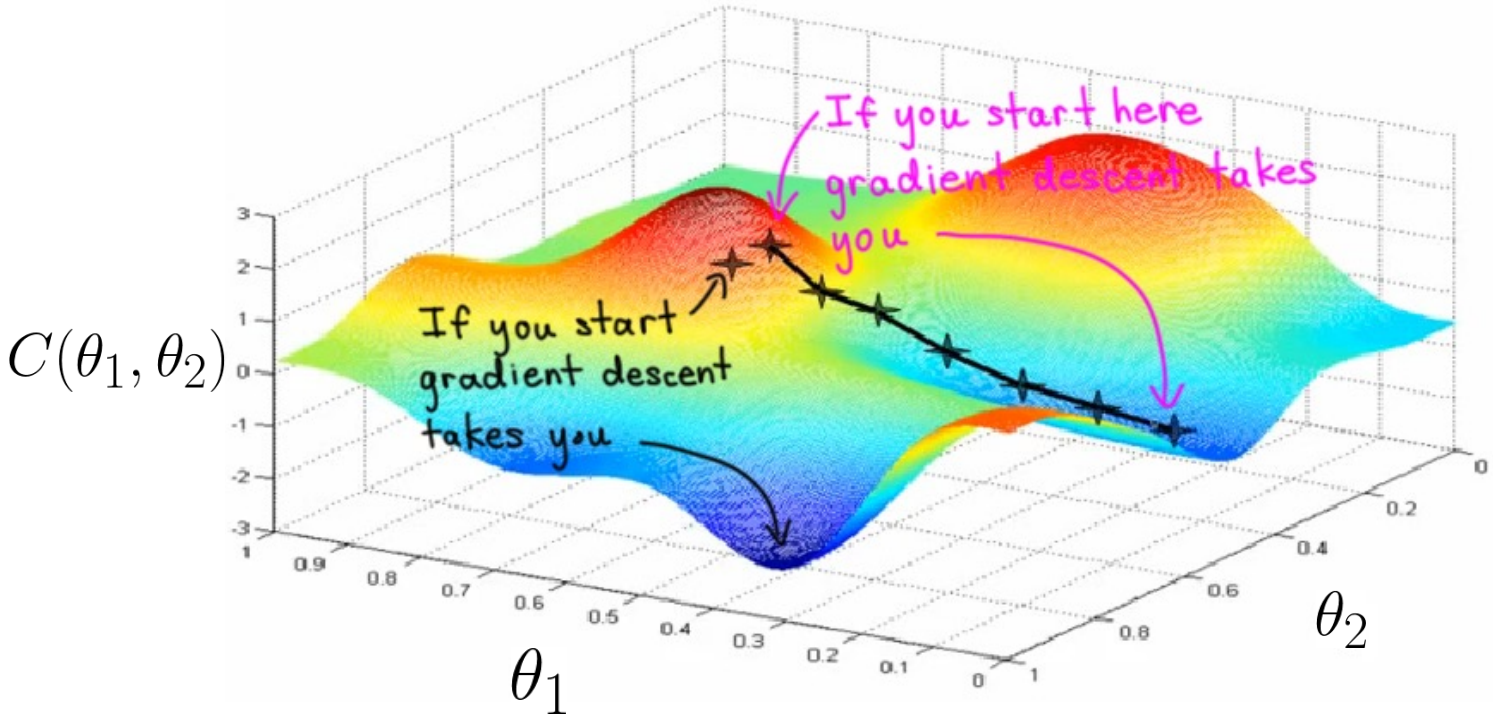
- Different initialization parameters may result in different trained models



Local optimum

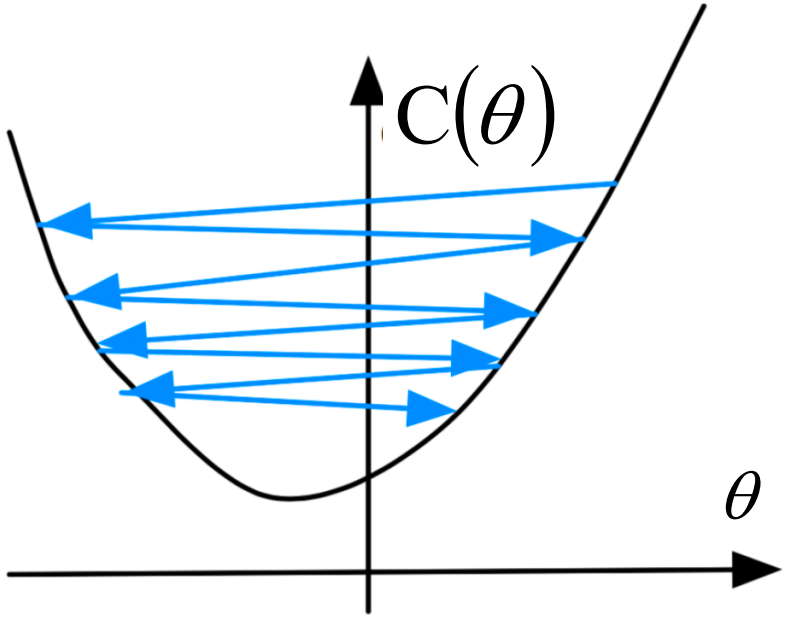
Practical Tips: Initialization

- Different initialization parameters may result in different trained models



Do not initialize the parameters equally → set them randomly

Practical Tips: Learning Rate



Too high learning rate

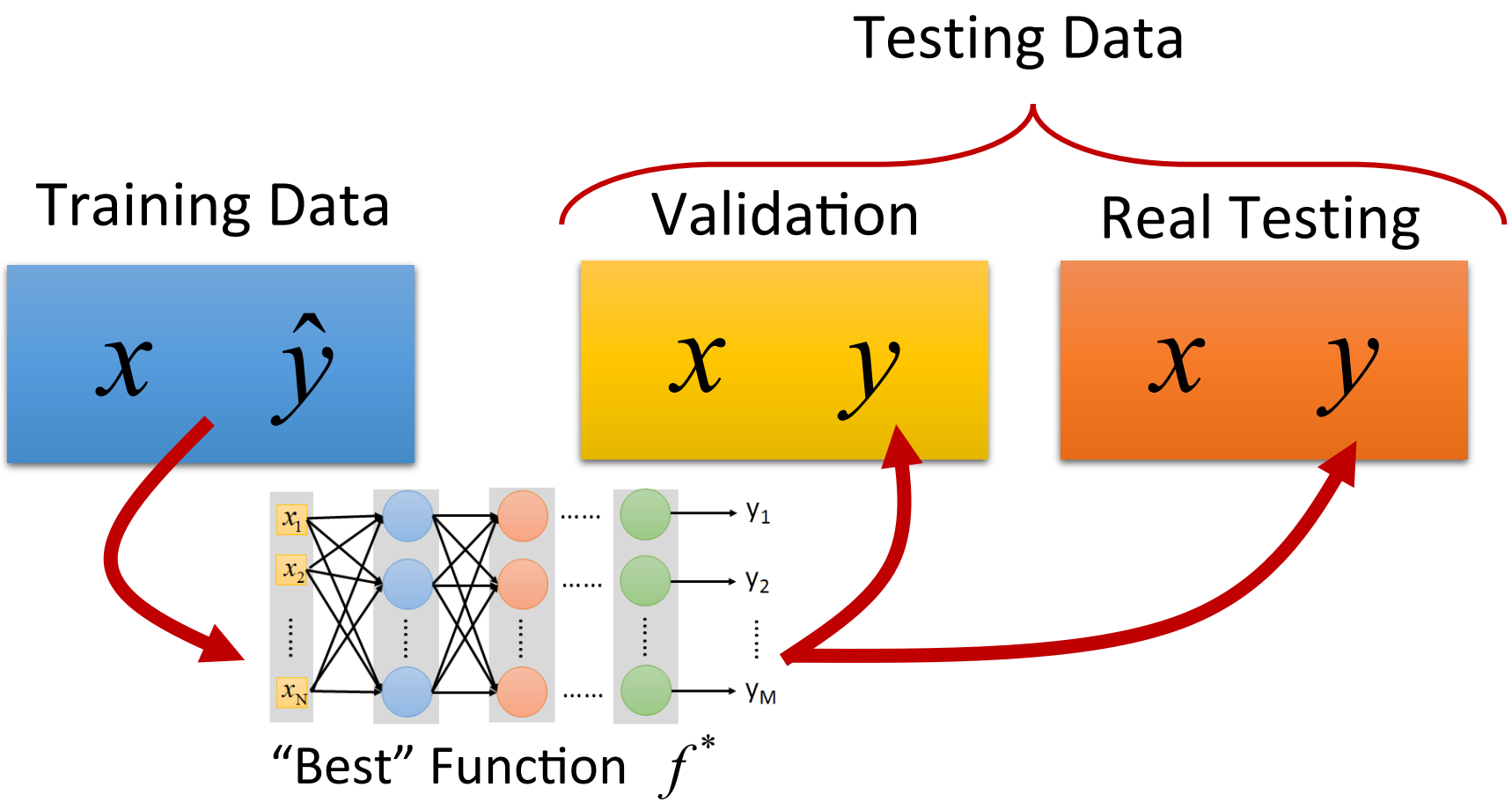
$$\theta^{i+1} \leftarrow \theta^{i} - \eta \nabla_{\theta} C(\theta^{i})$$

Learning rate should be set carefully

Tips for Mini-Batch Training

- Shuffle training samples before every epoch
  - the network might memorize the order you feed the examples
- Use a fixed batch size for every epoch
  - enable fast implementation of matrix multiplication for calculations
- Adapt the learning rate to the batch size
  - larger batch → smaller learning rate

Learning Recipe



Learning Recipe

Testing Data

Training Data

$x \quad \hat{y}$

Validation

$x \quad y$

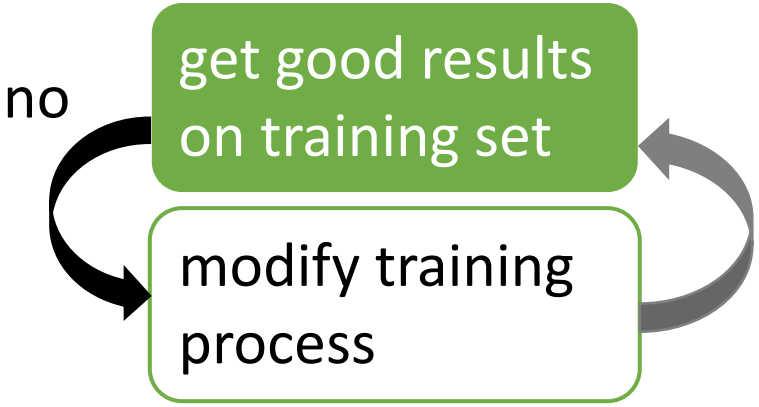
immediately observe the performance

Real Testing

$x \quad y$

Will not know the performance immediately

Learning Recipe

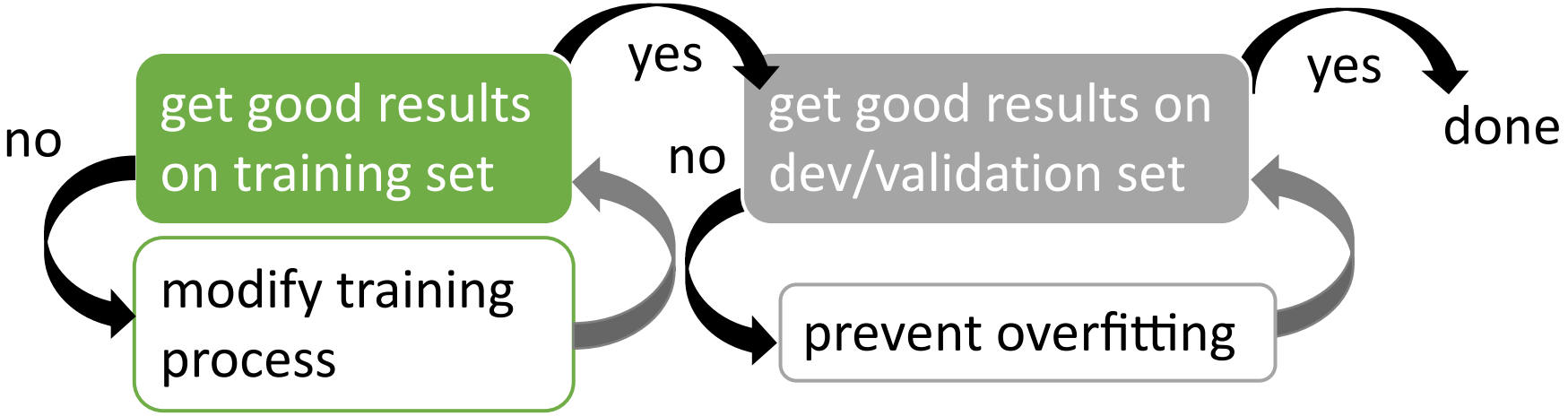


- Possible reasons
  - no good function exists: bad hypothesis function set

  → reconstruct the model architecture
  - cannot find a good function: local optima

  → change the training strategy

Learning Recipe



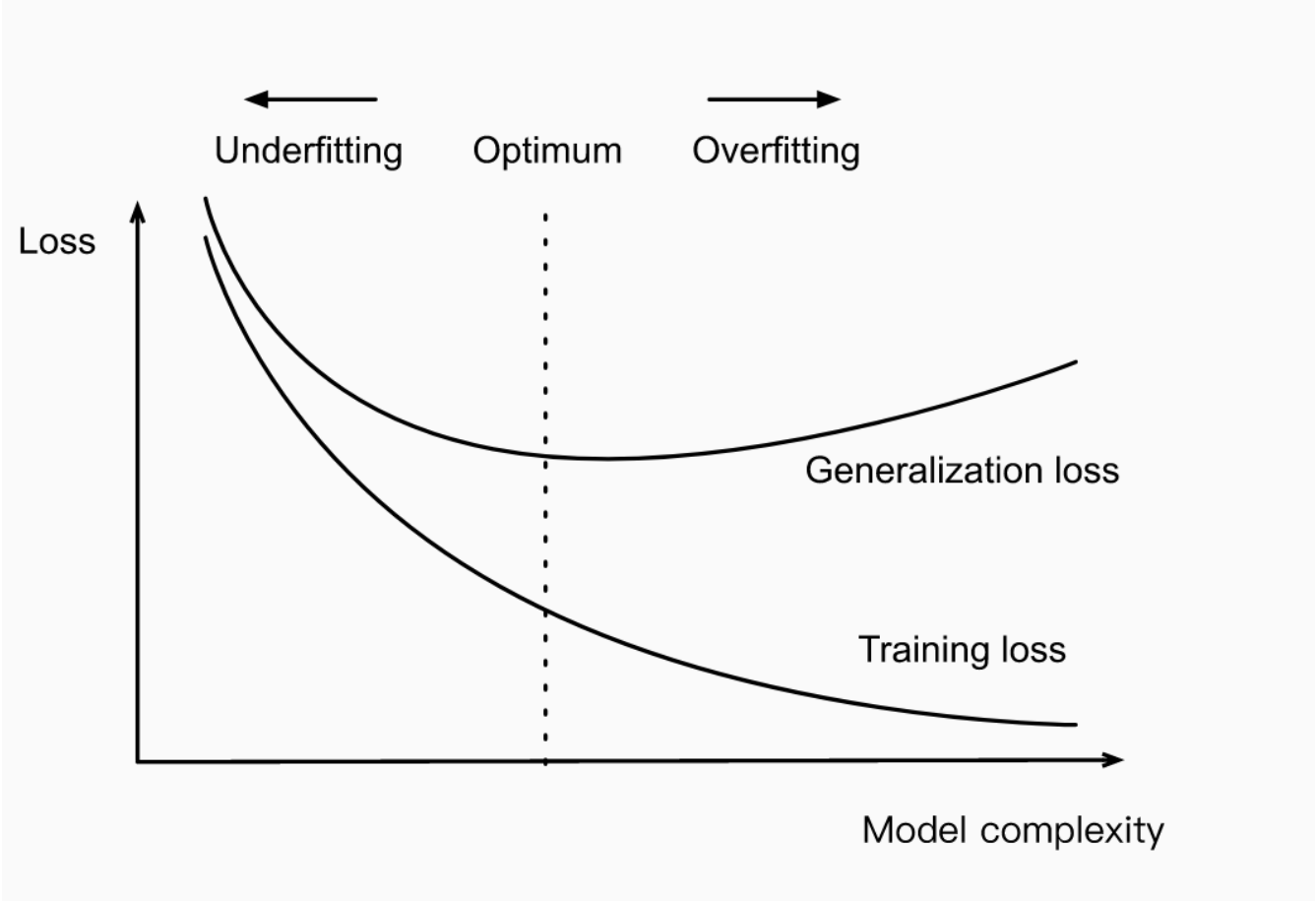
Better performance on training but worse performance on dev → overfitting

Generalization

- Training Error
  - *Statistic* calculated on the training dataset

- Generalization Error
  - you would see if you applied your model to an infinite stream of additional data examples drawn from the same underlying data distribution.
  - we can never calculate the generalization error exactly.
  - *Estimate* the generalization error

Polynomial Curve Fitting



Influence of model complexity on underfitting and overfitting

Overfitting

- Possible solutions
  - more training samples
  - some tips: dropout, etc.
  - Cross validation

Linear Neural Networks for Classification

- Regression is useful for “how much?” and “how many?” questions, where the target can take a real value.
- Many NLP problems are about “which one?”
  - Sentiment classification: positive, negative(, neutral)
  - Sentence boundary detection from text: is the punctuation mark defining a sentence boundary or not
  - What is the part-of-speech tag of the word “word” in this sentence? NOUN, VERB, etc.?
- We can represent target labels with one-hot encodings as well. For example, for sentiment classification:

$$y \in \{\underbrace{(1,0,0)}_{\text{Positive}}, \underbrace{(0,1,0)}_{\text{Negative}}, \underbrace{(0,0,1)}_{\text{Neutral}}\}.$$

Network Architecture for Multi-Class Problems

- To estimate the conditional probabilities associated with each class, we need a model with multiple outputs, one per class.
- As many linear functions as we have outputs.
- Example: Assume our input is represented with 4 features, then to compute the logits o_1, o_2, o_3, for each output, we need:

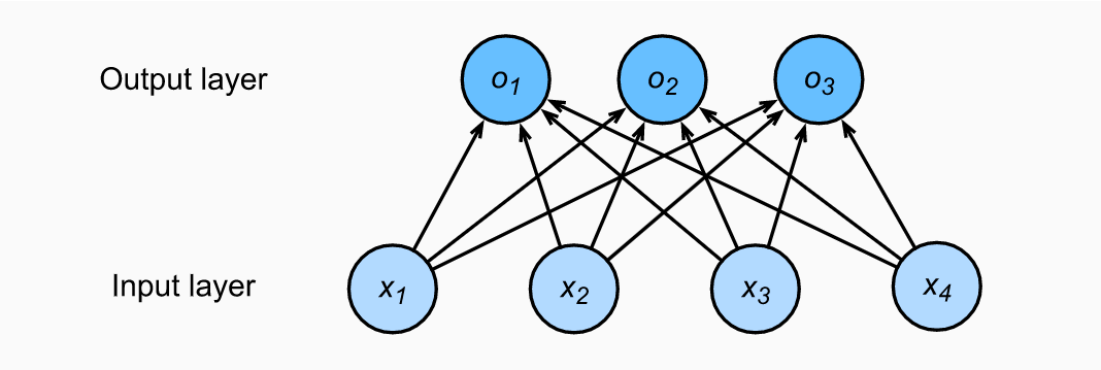
$$o_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1$$

$$o_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2$$

$$o_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3$$

$$\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

3 x 4 matrix



Softmax regression is a single-layer neural network.

Multi-Class Problems

- Interpret model outputs as probabilities and optimize our parameters to produce probabilities that maximize the likelihood of the observed data
- To generate predictions, we can set a threshold or choose the *argmax* of the predicted probabilities

- Interpret the logits o directly as our outputs of interest, however:
  - Nothing constrains these numbers to sum to 1.
  - Depending on the inputs, they can take negative values.

Softmax

- To estimate the conditional probabilities associated with each class, we need a model with multiple outputs, one per class.
- As many linear functions as we have outputs.
- Example: Assume our input is represented with 4 features, then to compute the logits o_1, o_2, o_3, for each output, we need:

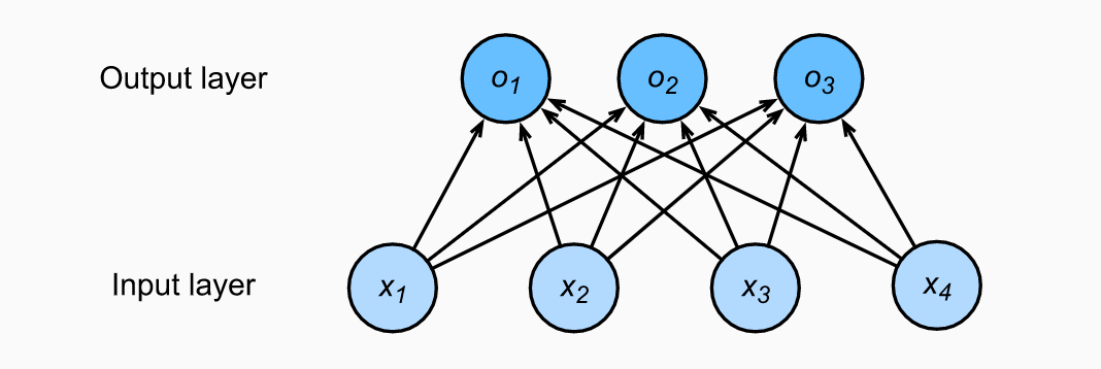
$$o_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1$$

$$o_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2$$

$$o_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3$$

$$\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

3 x 4 matrix



Softmax regression is a single-layer neural network.

Softmax

- Transforms logits such that they become nonnegative and sum to 1, while requiring that the model remains differentiable.

$$\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{o}) \quad \text{where} \quad \hat{y}_i = \frac{\exp(o_i)}{\sum_j \exp(o_j)}$$

- $\hat{y}_1+\hat{y}_2+\hat{y}_3 = 1$,
- $0 \leq \hat{y}_i \leq 1$ for all i, and
- The ordering of the logits has not changed, hence we can still pick the output using argmax!

Maximum Likelihood Estimation

- The softmax function gives us a vector $\hat{\mathbf{y}}$, interpreted as estimated conditional probabilities of each class given the input x, e.g., $\hat{y}_1 = P$ (y=cat|$\mathbf{x}$).

$$P(Y \mid X) = \prod_{i=1}^{n} P(y^{(i)} \mid x^{(i)})$$

$$-\log P(Y \mid X) = \sum_{i=1}^{n} -\log P(y^{(i)} \mid x^{(i)})$$

- Maximizing $P(Y|X)$ (and thus equivalently minimizing $-\log P(Y|X)$) corresponds to predicting the label well!

Cross-Entropy Loss and Softmax

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^{q} y_j \log \hat{y}_j.$$

q: the number of classes

- If we add o into the definition of the loss l and use the definition of the softmax we obtain:

$$\begin{aligned} l(\mathbf{y}, \hat{\mathbf{y}}) &= -\sum_{j=1}^{q} y_j \log \frac{\exp(o_j)}{\sum_{k=1}^{q} \exp(o_k)} \\ &= \sum_{j=1}^{q} y_j \log \sum_{k=1}^{q} \exp(o_k) - \sum_{j=1}^{q} y_j o_j \\ &= \log \sum_{k=1}^{q} \exp(o_k) - \sum_{j=1}^{q} y_j o_j. \end{aligned}$$

- If we compute its derivative with respect to o_j, we get:

$$\partial_{o_j} l(\mathbf{y}, \hat{\mathbf{y}}) = \frac{\exp(o_j)}{\sum_{k=1}^{q} \exp(o_k)} - y_j = \mathrm{softmax}(\mathbf{o})_j - y_j.$$

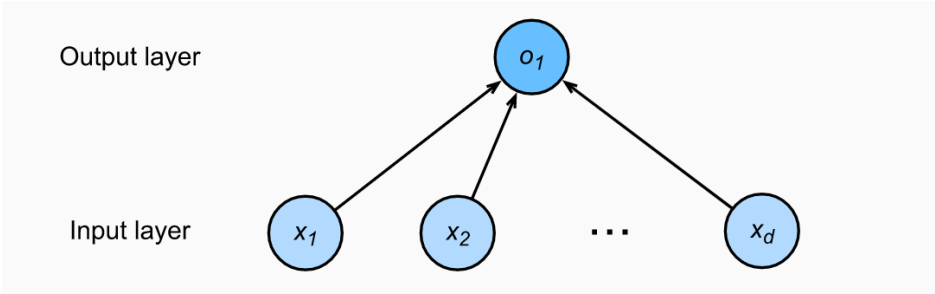
The gradient is the difference between the probability assigned to the true class by our model. This makes computing gradients very easy in practice!

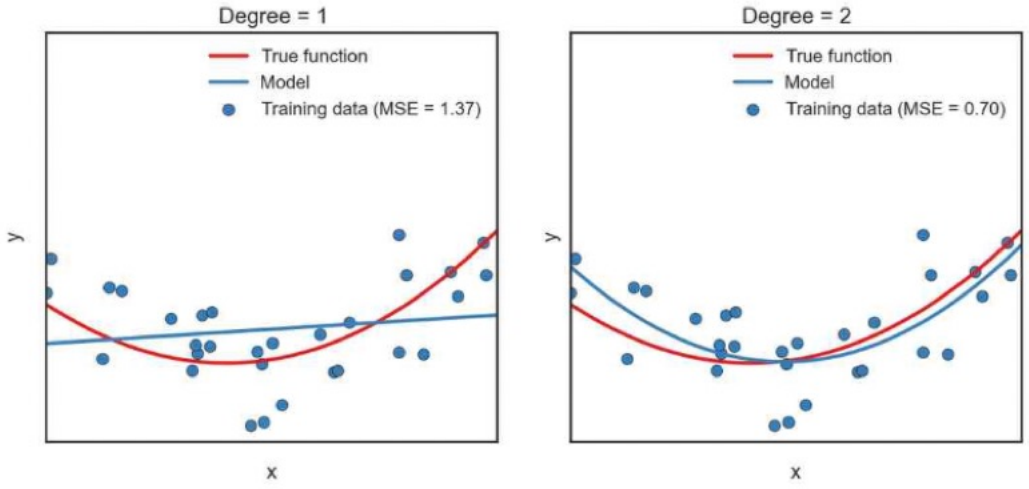
Limitations of Linear Models

- Linearity implies the weaker assumption of monotonicity
- increase in our feature must either always cause an increase in our model's output (if the corresponding weight is positive), or always cause a decrease in our model's output if the weight is negative.

Non-linearity

- Linear regression inputs directly to our outputs via a single linear transformation.
- But what if our data is not linear?



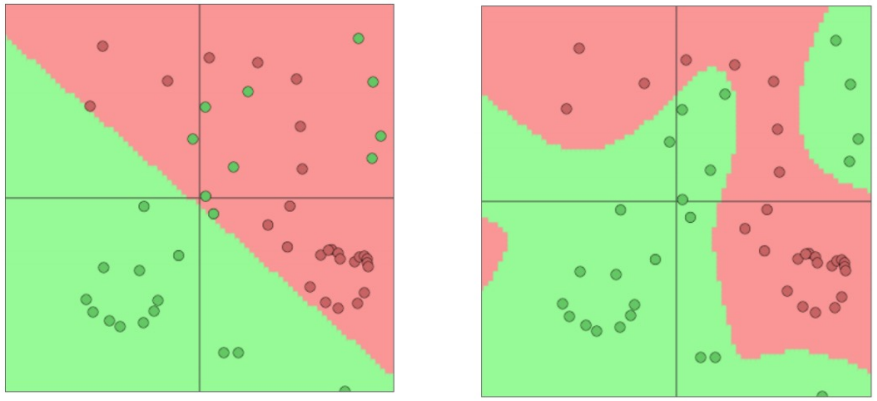


Why Non-Linearity?

- Function approximation
  - ***Without non-linearity***, deep neural networks work the same as linear transform

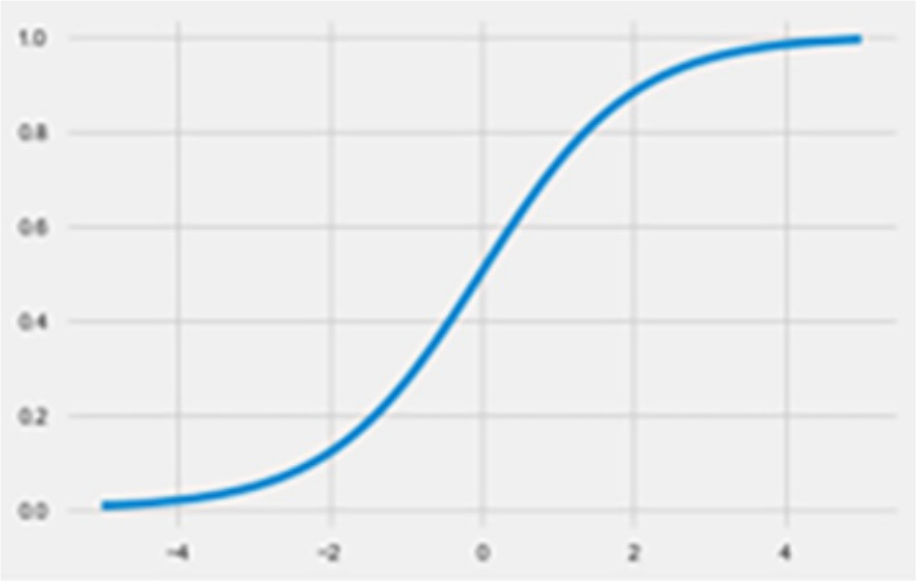
$$W_1(W_2 \cdot x) = (W_1 W_2)x = Wx$$

  - ***With non-linearity***, networks with more layers can approximate more complex functions



Activation Functions in PyTorch - Sigmoid

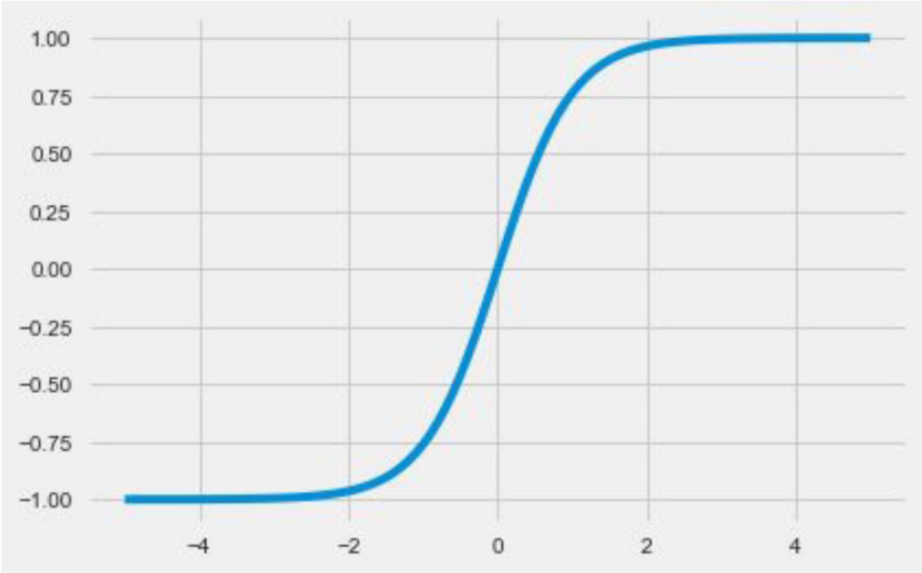
```
import torch
import matplotlib.pyplot as plt

x = torch.range(-5., 5., 0.1)
y = torch.sigmoid(x)
plt.plot(x.numpy(), y.numpy())
plt.show()
```



$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

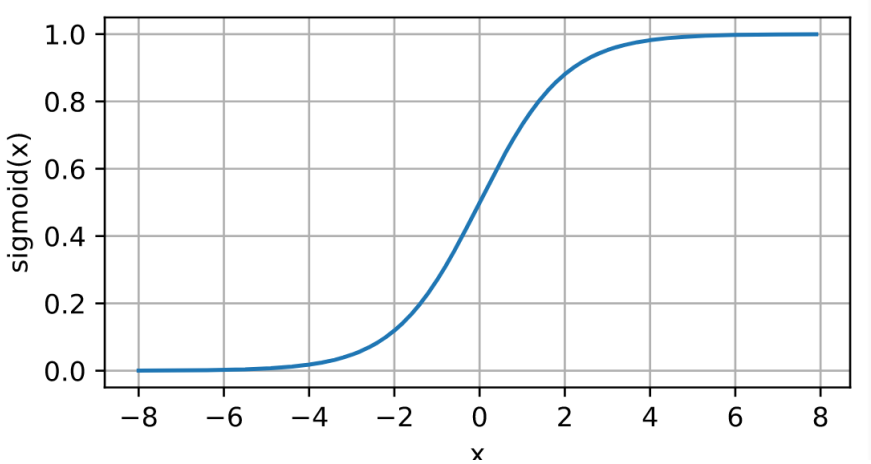
Activation Functions in PyTorch - Tanh

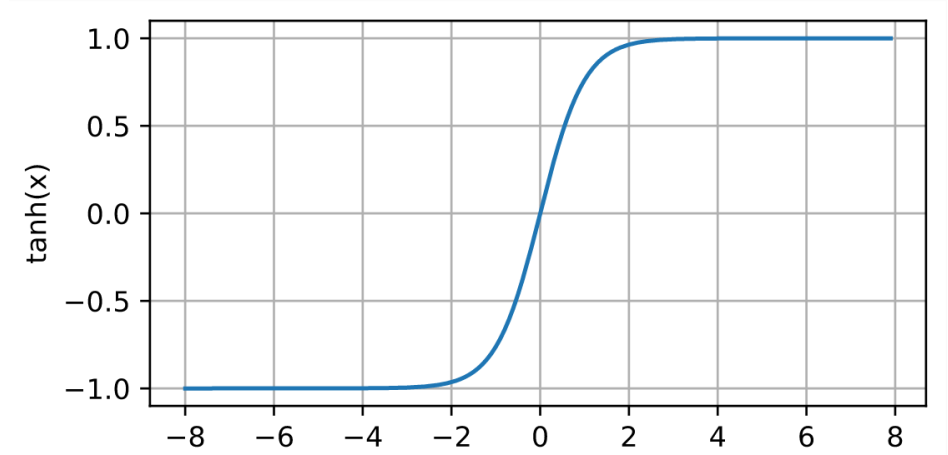
```
import torch
import matplotlib.pyplot
as plt

x = torch.range(-5., 5.,
0.1)
y = torch.tanh(x)
plt.plot(x.numpy(),
y.numpy())
plt.show()
```



$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

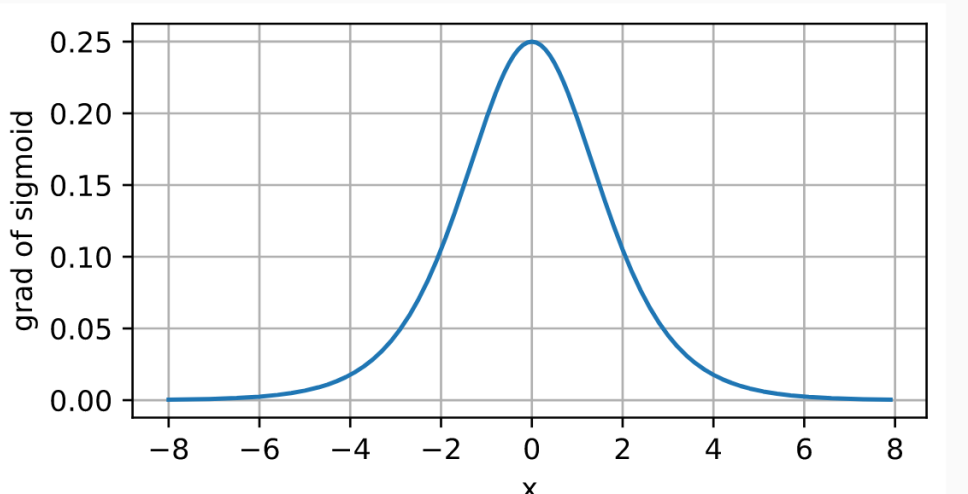
Vanishing Gradients

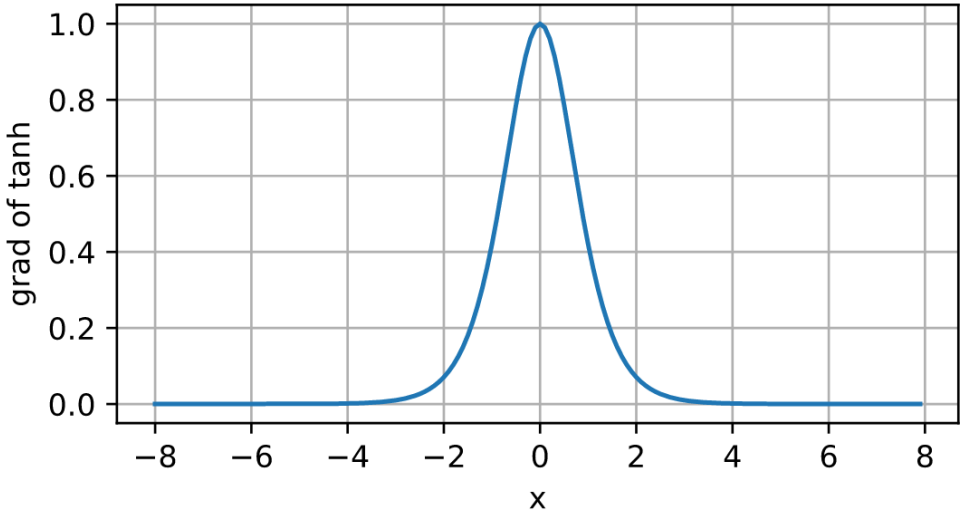




$$\frac{d}{dx}\text{sigmoid}(x) = \frac{\exp(-x)}{(1+\exp(-x))^2} = \text{sigmoid}(x)\,(1-\text{sigmoid}(x)).$$

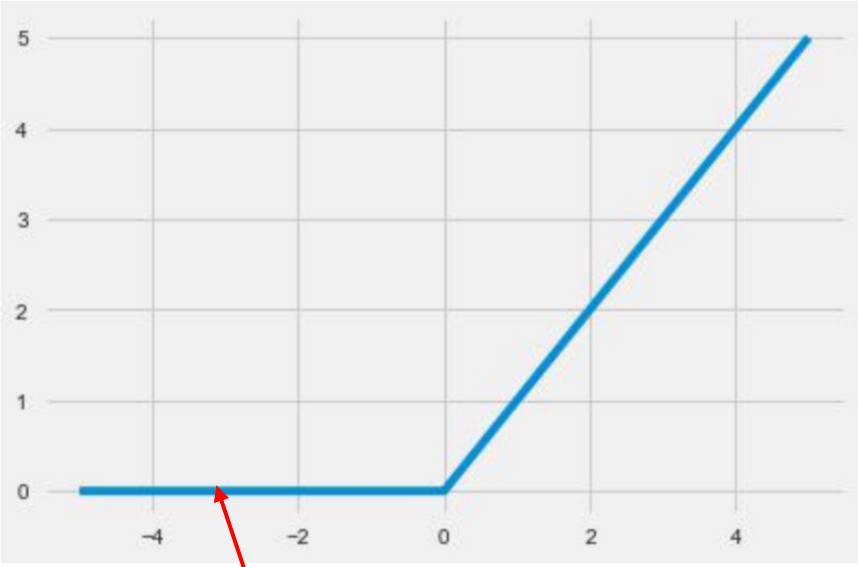
$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$$





Activation Functions in PyTorch - ReLU

```
import torch
import matplotlib.pyplot
as plt

relu = torch.nn.ReLU()
x = torch.range(-5., 5.,
0.1)
y = relu(x)

plt.plot(x.numpy(),
y.numpy())
plt.show()
```



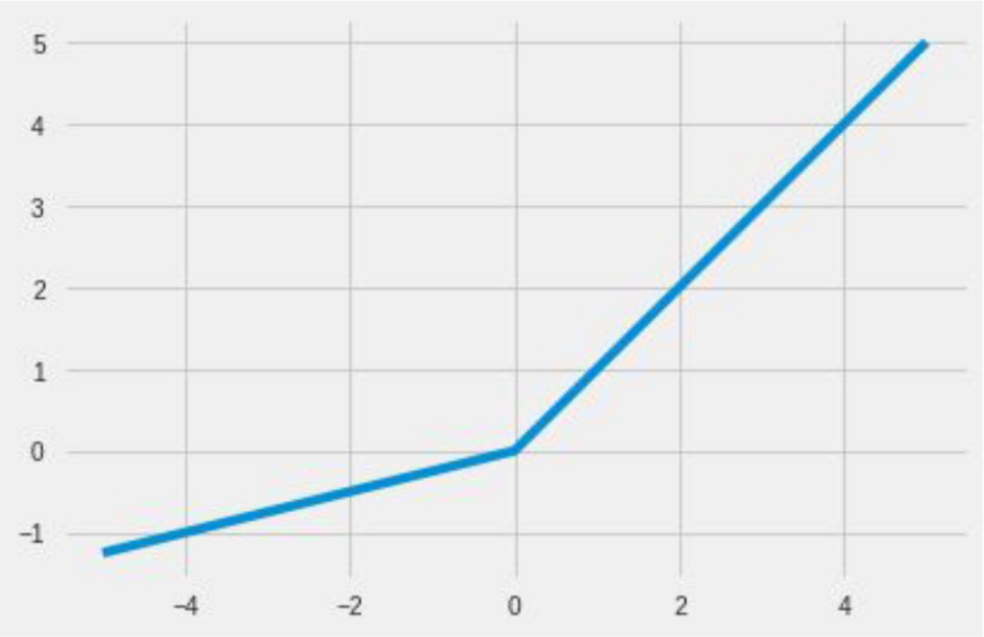
Clipping negative values to zero!

$$\mathrm{ReLU}(x) = \max(x, 0)$$

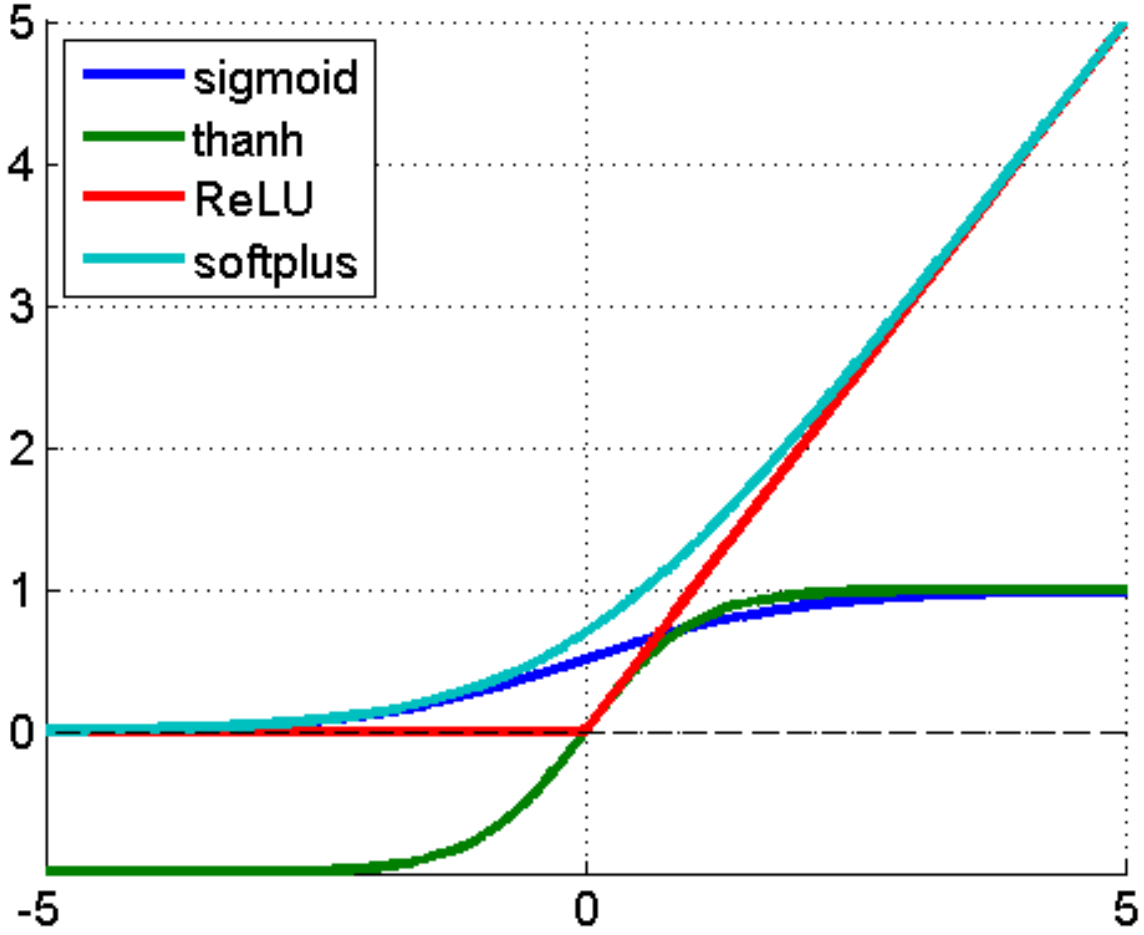
However, over time certain outputs in the network can simply become zero and never revive again.

Activation Functions in PyTorch - PReLU

```
import torch
import matplotlib.pyplot as plt

prelu =
torch.nn.PReLU(num_parameters=1)
x = torch.range(-5., 5., 0.1)
y = prelu(x)

plt.plot(x.numpy(), y.numpy())
plt.show()
```



Non-Linear Activation Functions - Summary



Activation Functions in PyTorch - Softmax

```python
import torch
import torch.nn as nn

softmax = nn.Softmax(dim=1)
x_input = torch.randn(1,4)
y_output = softmax(x_input)
print(x_input)
print(y_output)
print(torch.sum(y_output, dim=1))
```

```
tensor([[ 0.4453, -0.8326, -0.4668,  0.5607]])
tensor([[0.3568, 0.0994, 0.1433, 0.4005]])
tensor([1.0000])
```

Loss Functions in PyTorch – Mean Squared Error Loss

```python
import torch
import torch.nn as nn

mse_loss = nn.MSELoss()
outputs = torch.randn(3, 5, requires_grad=True)
targets = torch.randn(3, 5)
loss = mse_loss(outputs, targets)
print(outputs)
print(targets)
print(loss)
```

```
tensor([[ 0.3472, -0.2246,  0.1670, -0.2578,  1.5375],
        [ 0.1875, -0.9442,  0.2554,  1.4726,  1.3097],
        [ 1.6956, -0.2329,  0.7398,  0.6420,  2.1655]], requires_grad=True)
tensor([[ 4.4099e-01,  3.5975e-01, -3.3145e-02,  1.6507e+00, -9.6259e-02],
        [-1.3071e+00,  2.6525e-04, -3.9299e-01,  1.3937e+00,  1.0394e+00],
        [-9.4118e-02,  7.5348e-01, -2.3449e-01, -9.0545e-01, -1.3017e+00]])
tensor(1.9913, grad_fn=<MseLossBackward>)
```

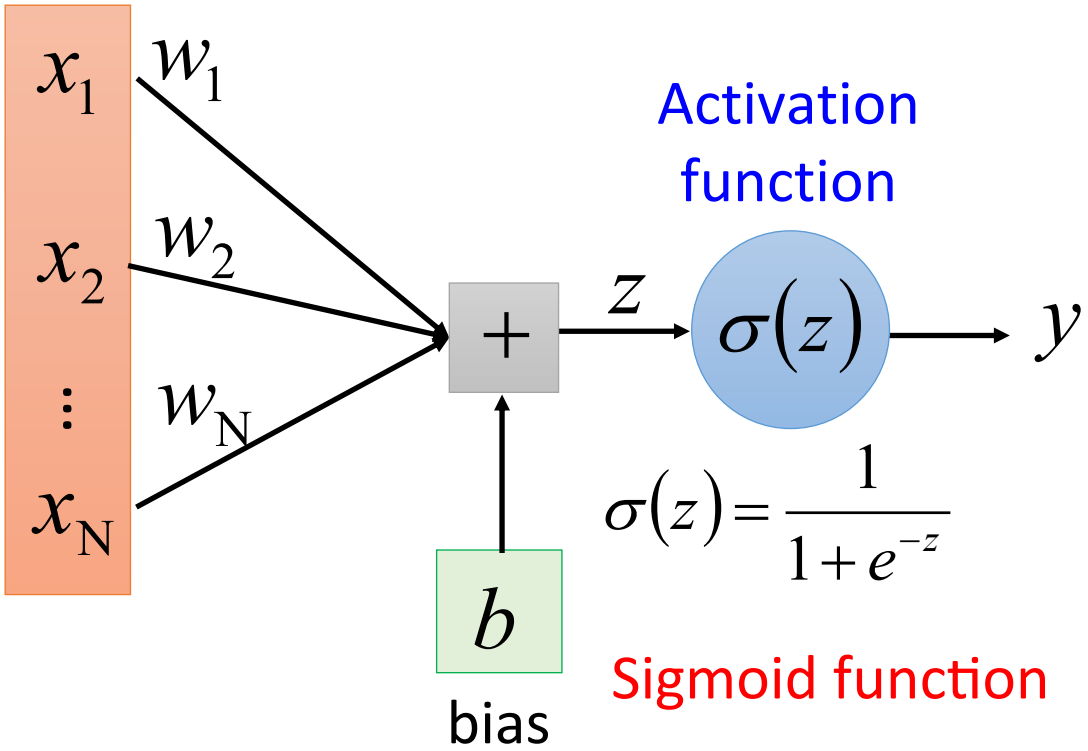
Loss Functions in PyTorch-Categorical Cross-Entropy Loss

```python
import torch
import torch.nn as nn

ce_loss = nn.CrossEntropyLoss()
outputs = torch.randn(3, 5, requires_grad=True)
targets = torch.tensor([1, 0, 3], dtype=torch.int64)
loss = ce_loss(outputs, targets)
print(outputs)
print(targets)
print(loss)
```

```
tensor([[ 1.2979,  0.5740, -1.5945, -2.4253,  0.1673],
        [ 1.0170,  0.3193,  0.4528,  1.3819, -1.2214],
        [-0.3760,  0.6830,  0.5004, -1.6647,  0.2250]], requires_grad=True)
tensor([1, 0, 3])
tensor(2.0196, grad_fn=<NllLossBackward>)
```

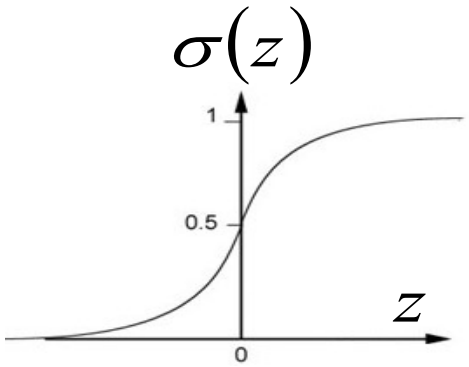
- typically used in a multiclass classification, where the outputs are interpreted as predictions of class membership probabilities
- Nn.BCELoss() for the binary

Perceptron – A single neuron



$$\sigma(z) = \frac{1}{1+e^{-z}}$$

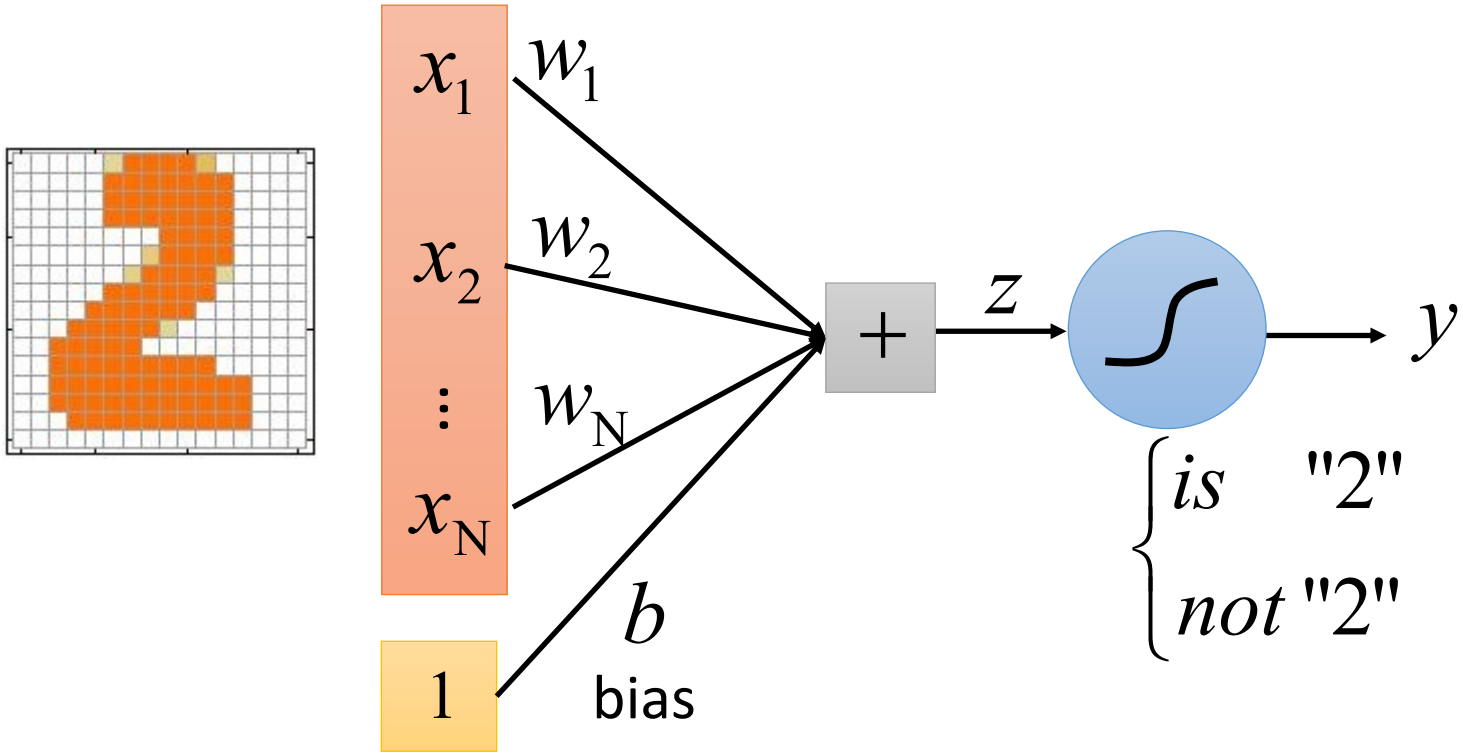
Sigmoid function



Each neuron is a very simple function

Perceptron - A Single Neuron

$$f : R^N \rightarrow R^M$$



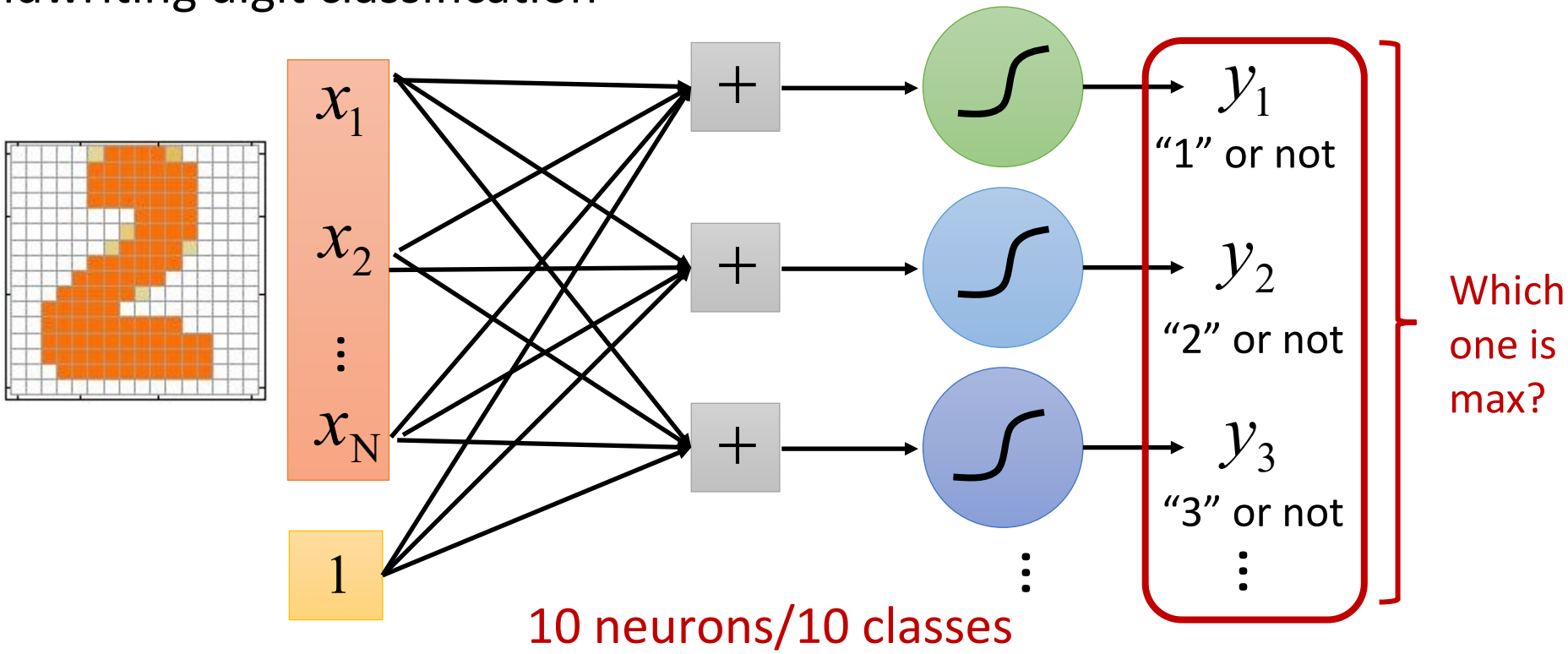
$$\begin{cases} is \quad "2" & y \geq 0.5 \\ not\ "2" & y < 0.5 \end{cases}$$

A single neuron can only handle binary classification

A Layer of Neurons

$$f : R^N \rightarrow R^M$$

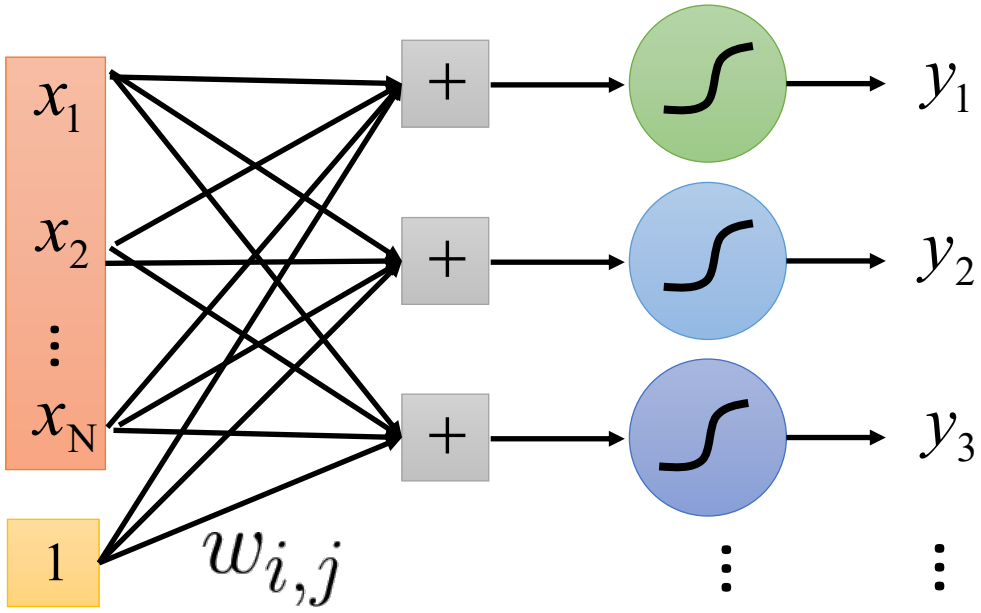
- Handwriting digit classification

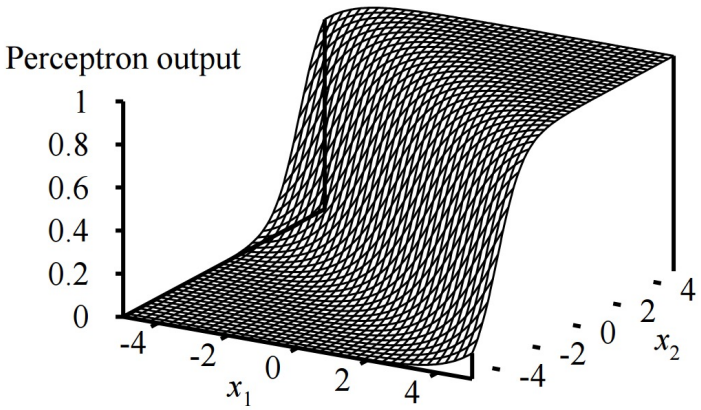


A layer of neurons can handle multiple possible output,
and the result depends on the max one

A Layer of Neurons – Perceptron

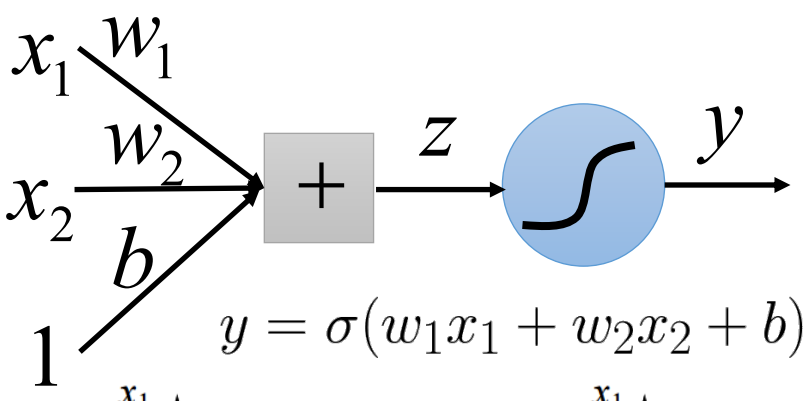
- Output units all operate separately – no shared weights





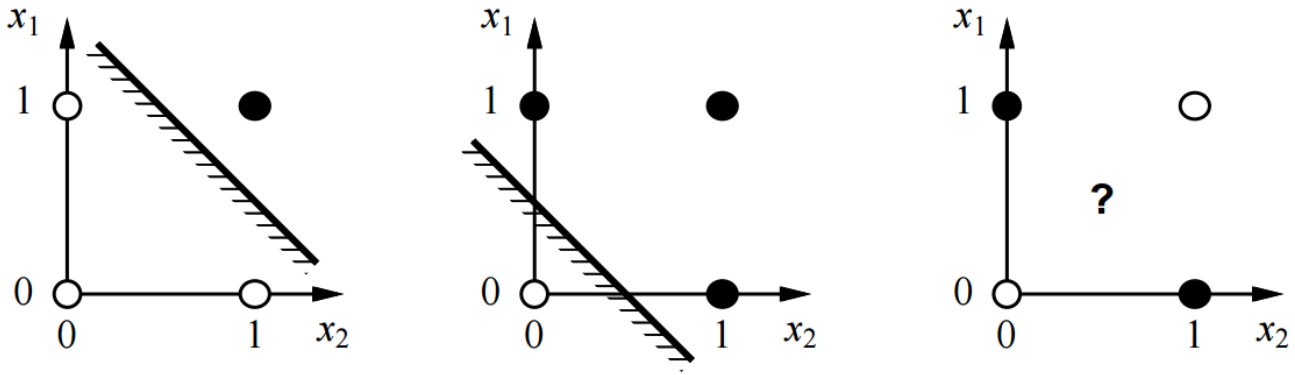
Adjusting weights moves the location, orientation, and steepness of cliff

Expression of Perceptron



$$y = \sigma(w_1 x_1 + w_2 x_2 + b)$$

$$\begin{cases} \text{yes} & y \geq \theta \text{ (threshold)} \\ \text{no} & y < \theta \end{cases}$$



(a) x_1 **and** x_2 (b) x_1 **or** x_2 (c) x_1 **xor** x_2

A perceptron can represent AND, OR, NOT, etc., but not XOR → linear separator

Next Class

- Review of Possible Topics for Final Projects
- Computational Graphs
- Regularization and Drop out
- Backpropagation and Gradient descent